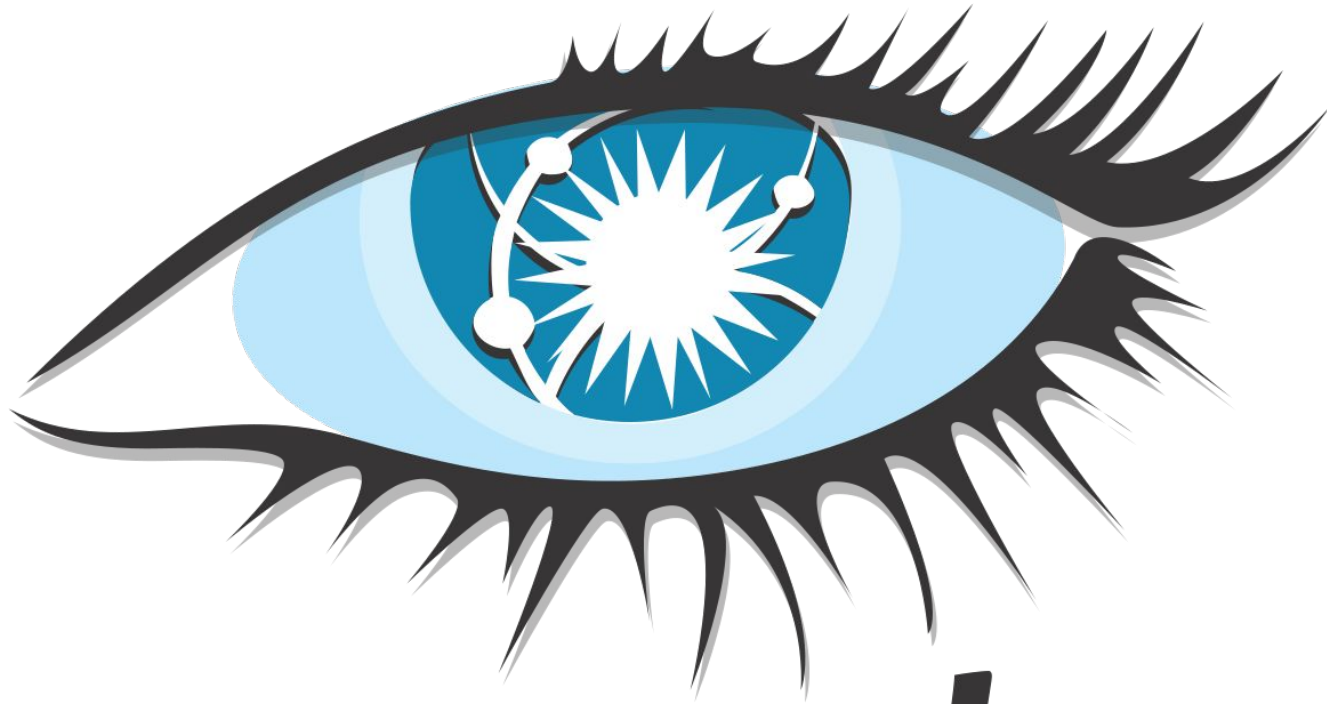




cassandra

What is NoSQL Database?

NoSQL databases, also known as "**non-SQL**" or "**not only SQL**", are databases that provide a mechanism to store and retrieve data modeled in ways other than the tabular format used in relational databases. They are typically used in large-scale or real-time web applications where the ability to scale quickly and handle large, diverse types of data is critical.

Here are some key characteristics and features of NoSQL databases:

1. *Schema-less*: NoSQL databases do not require a fixed schema, which gives you the flexibility to store different types of data entities together.
2. *Scalability*: NoSQL databases are designed to expand easily to handle more traffic. They are horizontally scalable, meaning you can add more servers to handle larger amounts of data and higher loads.
3. *Diverse Data Models*: NoSQL databases support a variety of data models including key-value pairs, wide-column, graph, or document. This flexibility allows them to handle diverse types of data and complex data structures.
4. *Distributed Architecture*: Many NoSQL databases are designed with a distributed architecture, which can improve fault tolerance and data availability.
5. *Performance*: Without the need for data relationships and joins as in relational databases, NoSQL data offer high-performance reads and writes.

Types of NoSQL Databases

NoSQL databases are categorized into four basic types based on the way they organize data. Let's go through each type and provide examples:

Document Databases: These store data in documents similar to JSON (JavaScript Object Notation) objects. Each document contains pairs of fields and values, and the values can typically be a variety of types including strings, numbers, booleans, arrays, or objects. Each document is unique and can contain different data from other documents in the collection. This structure makes document databases flexible and adaptable to various data models.

Example: MongoDB, CouchDB.

Key-Value Databases: These are the simplest type of NoSQL databases. Every single item in the database is stored as an attribute name (or 'key') and its value. The main advantage of a key-value store is the ability to read and write operations using a simple key. This type of NoSQL database is typically used for caching and session management.

Example: Redis, Amazon DynamoDB.

Types of NoSQL Databases

Wide-Column Stores: These databases store data in tables, rows, and dynamic columns. Wide-column stores offer high performance and a highly scalable architecture. They're ideal for analyzing large datasets and are capable of storing vast amounts of data (Big Data).

Example: Apache Cassandra, Google BigTable.

Graph Databases: These are used to store data whose relations are best represented as a graph. Each node of the graph represents an entity, and the relationship between nodes is stored directly, which allows the data to be retrieved in one operation. They're ideal for storing data with complex relationships, like social networks or a network of IoT devices.

Example: Neo4j, Amazon Neptune

Difference between Transactional & NoSQL Database

Feature	Transactional (Relational) Databases	NoSQL Databases
Data Structure	Organized into tables, with relationships defined between them.	Different types of data models including key-value, document, columnar, and graph.
Schema	Pre-defined schema required. The structure of data needs to be defined before use.	Schema-less. The structure of data can be altered dynamically.
Scaling	Usually vertically scalable. You can increase the power (CPU, RAM, SSD) of a single server.	Horizontally scalable. You can add more servers in your NoSQL database infrastructure to handle more traffic.
Transactions	Full ACID (Atomicity, Consistency, Isolation, Durability) compliance to ensure reliable processing of transactions.	Generally, they do not provide full ACID compliance but offer eventual consistency. Some NoSQL databases, however, do provide ACID compliance.
Complex Queries	Best suited for complex queries as they support JOINS and other complex operations.	Not as powerful for complex queries because NoSQL databases don't have standard interfaces to perform complex queries, and the queries themselves can be complex to design.
Speed and Performance	High transactional performance, but for read-heavy workloads, performance can degrade due to table joins.	High-performance reads and writes. NoSQL databases are typically optimized for specific data models and access patterns that enable higher performance.
Reliability	Highly reliable and widely adopted in industries requiring strict consistency models.	Dependability varies across different NoSQL database types. However, their distributed architectures improve fault tolerance.
Use Case Examples	Transactional systems, ERP, CRM, etc.	Real-time applications, content management, IoT applications, etc.

NoSQL Databases are Good fit for Analytical Queries?



While NoSQL databases can handle certain analytical tasks, their primary purpose is not for heavy analytical queries. Traditional relational databases and data warehousing solutions, such as Hive, Redshift, Snowflake or BigQuery, are often better suited for complex analytical queries due to their ability to handle operations like joins and aggregations more efficiently.

NoSQL Databases in BigData Ecosystem

The strength of NoSQL databases lies in their flexibility, scalability, and speed for certain types of workloads, making them ideal for specific use-cases in the Big Data ecosystem:

1. **Handling Large Volumes of Data at Speed:** NoSQL databases are designed to scale horizontally across many servers, which enables them to handle large volumes of data at high speed. This is particularly useful for applications that need real-time read/write operations on Big Data.
2. **Variety of Data Formats:** NoSQL databases can handle a wide variety of data types (structured, semi-structured, unstructured), making them ideal for Big Data scenarios where data formats are diverse and evolving.
3. **Fault Tolerance and Geographic Distribution:** NoSQL databases have a distributed architecture that provides high availability and fault tolerance, critical for applications operating on Big Data.
4. **Real-time Applications:** Many Big Data applications require real-time or near-real-time functionality. NoSQL databases, with their high-speed read/write capabilities and ability to handle high volumes of data, are often used for real-time analytics, IoT data, and other similar scenarios.

That said, the choice between SQL, NoSQL, or other database technologies should be based on the specific needs of the use-case at hand.

CAP Theorem

The CAP theorem is a concept that a distributed computing system is unable to simultaneously provide all three of the following guarantees:

1. **Consistency (C):** Every read from the system receives the most recent write or an error. This implies that all nodes see the same data at the same time. It's the idea that you're always reading fresh data.
2. **Availability (A):** Every request receives a (non-error) response, without the guarantee that it contains the most recent write. It's the idea that you can always read or write data, even if it's not the most current data.
3. **Partition Tolerance (P):** The system continues to operate despite an arbitrary number of network or message failures (dropped, delayed, scrambled messages). It's the idea that the system continues to function even when network failures occur between nodes.

CAP Theorem

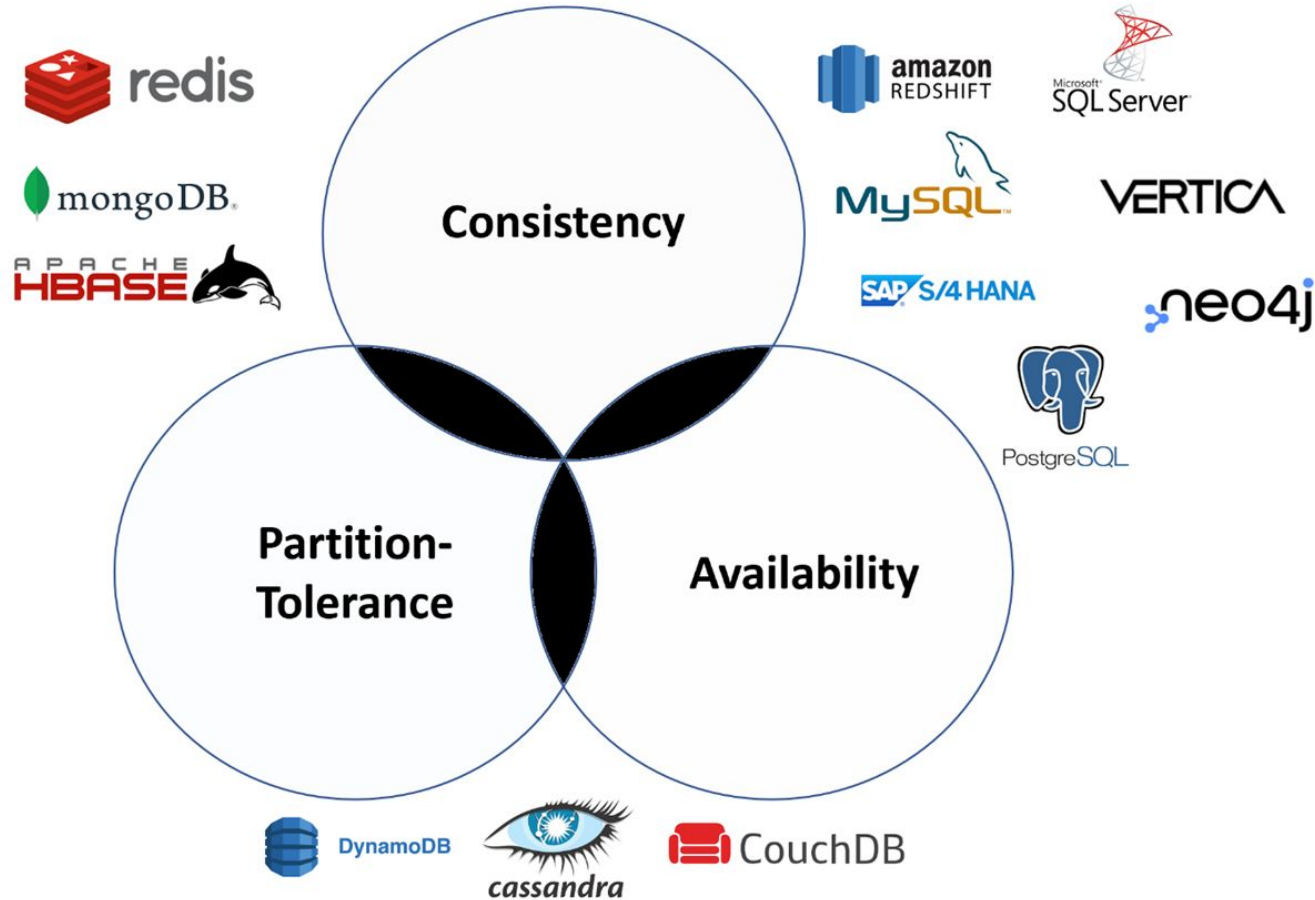
Now, the key aspect of the CAP theorem, proposed by computer scientist Eric Brewer, is that a distributed system can satisfy any two of these three guarantees at the same time, but not all three. Hence the term "CAP" - Consistency, Availability, and Partition tolerance.

Here's how the three dichotomies look like:

- **CA (Consistent and Available)** systems prioritize data consistency and system availability but cannot tolerate network partitions. In such a system, if there is a partition between nodes, the system won't work as it doesn't support partition tolerance.
- **CP (Consistent and Partition-tolerant)** systems prioritize data consistency and partition tolerance. If a network partition occurs, the system sacrifices availability to ensure data consistency across all nodes.
- **AP (Available and Partition-tolerant)** systems prioritize system availability and partition tolerance. If a network partition occurs, all nodes may not immediately reflect the same data, but the system remains available.

Remember, real-world systems must tolerate network partitions (P), so the practical choice is between consistency and availability (A) when partitions occur.

CAP Theorem



- **Open-Source Database System:** Cassandra is an open-source distributed database management system designed to handle large amounts of data across many commodity servers.
- **Wide-Column Store:** Cassandra falls under the wide-column store category of NoSQL databases. It arranges data by columns, allowing for great flexibility and scalability.
- **Highly Scalable:** Cassandra is designed to handle big data through its distributed architecture. It can scale horizontally to accommodate more data by simply adding more nodes to the system.
- **Fault-Tolerant:** It provides high availability and fault tolerance with no single point of failure. If a node fails, data can still be accessed from other nodes.
- **Decentralized:** Every node in the system has the same role with no master-slave dichotomy. This contributes to its robustness and resilience.
- **CAP Theorem:** In the context of CAP Theorem, Cassandra prioritizes Availability and Partition tolerance (AP) over Consistency. It provides eventual consistency.
- **Use Cases:** It's often used in applications where large data volumes are expected and a highly scalable, reliable system is required, such as in IoT, messaging systems, and fraud detection systems.

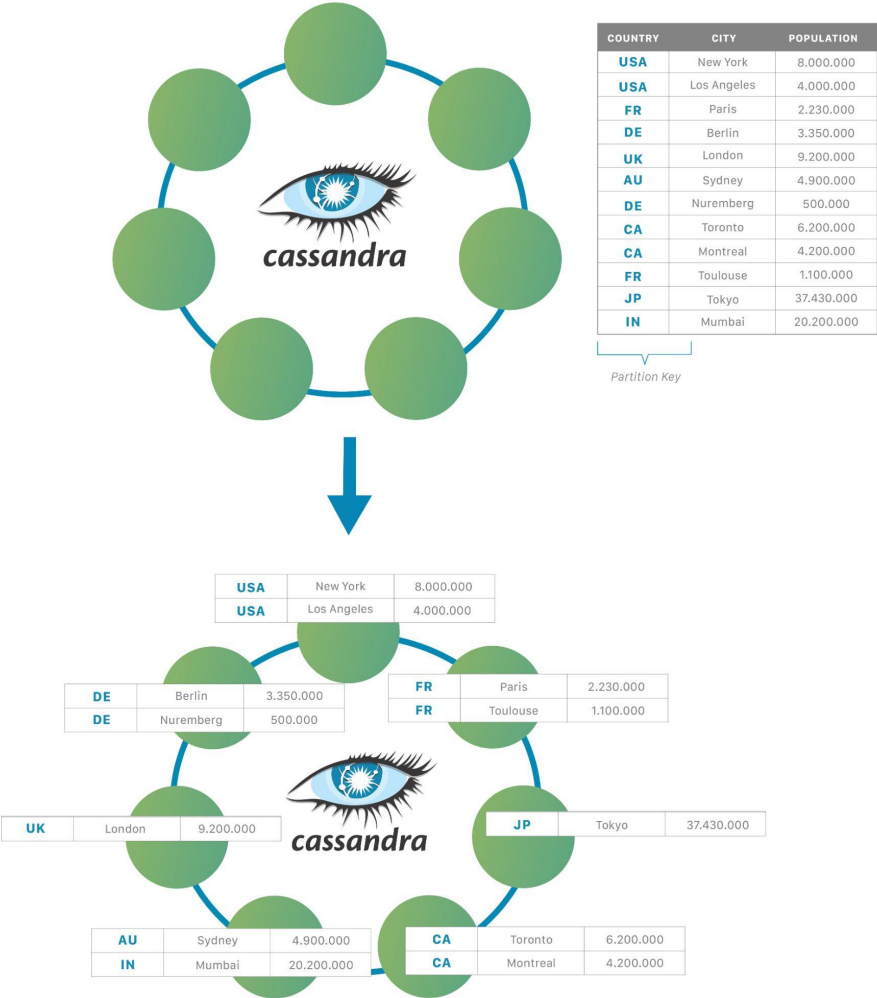
Cassandra is a good fit for??

Apache Cassandra is ideal for scenarios that need high availability, scalability, and quick data processing. It's best for handling **time-series** data, powering **messaging** systems, managing product **catalogs** and **playlists**, supporting **recommendation** systems, and running **fraud detection** systems.

Cassandra doesn't support JOINS

Cassandra doesn't support JOIN operations or other complex queries that relational databases can handle. JOIN operations typically involve merging data from multiple tables, which can be computationally expensive and slow, especially with large volumes of data. This goes against Cassandra's goal of providing high-speed, real-time performance.

Cassandra Architecture

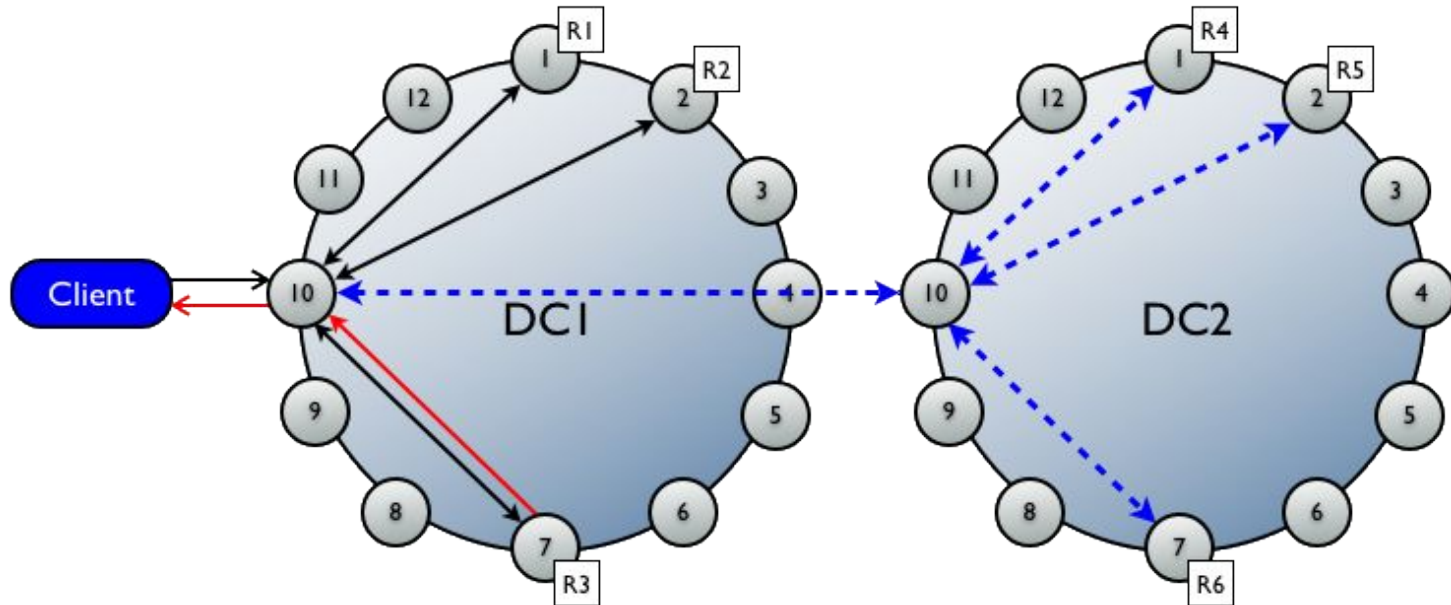


Features of Cassandra

- No single point of failure.
- Massive scalability .
- Possibility to add nodes any time.
- Highly distributed.
- High performance.
- No masters and slaves (Peer to peer).
- Ring type architecture
- Automatic data distribution across all nodes
- Replication of data across nodes
- Data kept in memory and written to disk in a lazy fashion
- Hash values of the keys are used to distribute data among nodes

Main Components of Cassandra

- **Node** – It is the place where data is stored.
- **Data center** – It is a collection of related nodes.
- **Cluster** – A cluster is a component that contains one or more data centers.



Cassandra organizes data into partitions. This is a common concept of distributed data systems. All the data is distributed into chunks called partitions. Partitioning is very important for performance and scalability.

- **Partition Key**: The partition key is responsible for distributing data among nodes. In Cassandra, we can only access data from the partitioning key.
- **Token**: A token in Cassandra is essentially a hash value. When data is written to Cassandra, it is hashed based on its partition key to produce a token value. This token value determines where the data should be stored. In this way, each piece of data is assigned a specific token. By default, a token is a **64-bit integer**. Therefore, the possible range for tokens is from **-2^{63} to $2^{63}-1$**
- **Token Range**: Each node in the cluster is responsible for a segment of the token ring, this segment is referred to as the **"token range"**. The token range defines the set of data that the node is responsible for. When a node owns a token range, it is responsible for storing all data that has a token value within that range.
- **Partitioners**: The partitioner determines how data is distributed across the nodes in a Cassandra cluster. Basically, a partitioner is a hash function to determine the token value by hashing the partition key of a row's data. Then, this partition key token is used to determine and distribute the row data within the ring. The **Murmur3Partitioner** is the default partitioner since Cassandra version 1.2. It uses the **MurmurHash** function that creates a 64-bit hash of the partition key.

Data Partitioning & Token

Car Name	Engine Capacity	Color	NoOfDoors
BMW	1000cc	Red	2
BMW	1500cc	Red	2
BMW	1500cc	Blue	2
BMW	1500cc	Red	4
BMW	1500cc	Blue	4
Toyata	1000cc	White	4
Toyata	1000cc	Black	4
Toyata	1500cc	Red	4
Audi	2000cc	Black	2
Audi	1500cc	Yellow	2
Benz	2000cc	White	2
Benz	2000cc	Red	2

In above table, **Car Name** is a partitioning key. When data is coming to the cassandra, it gets token of that incoming data by providing partitioning key to the hash function.

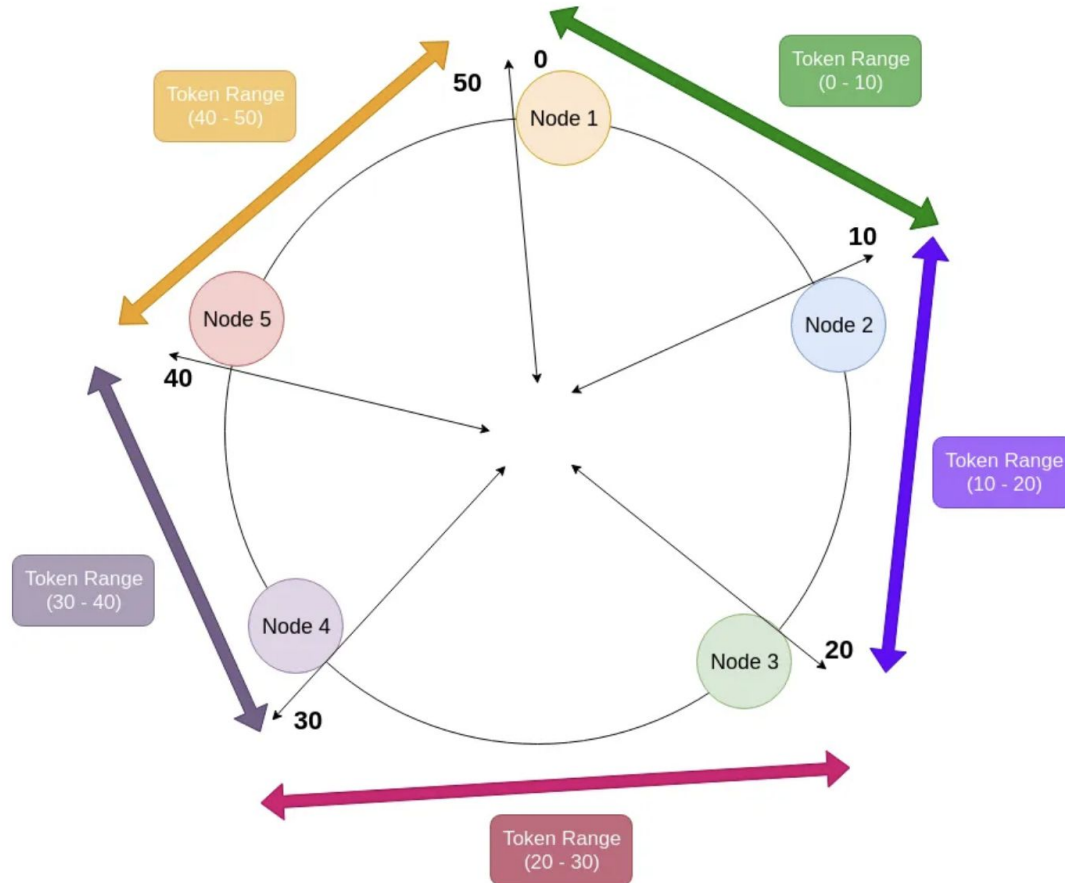
BMW -> Hash Function -> 9

Toyata -> Hash Function -> 17

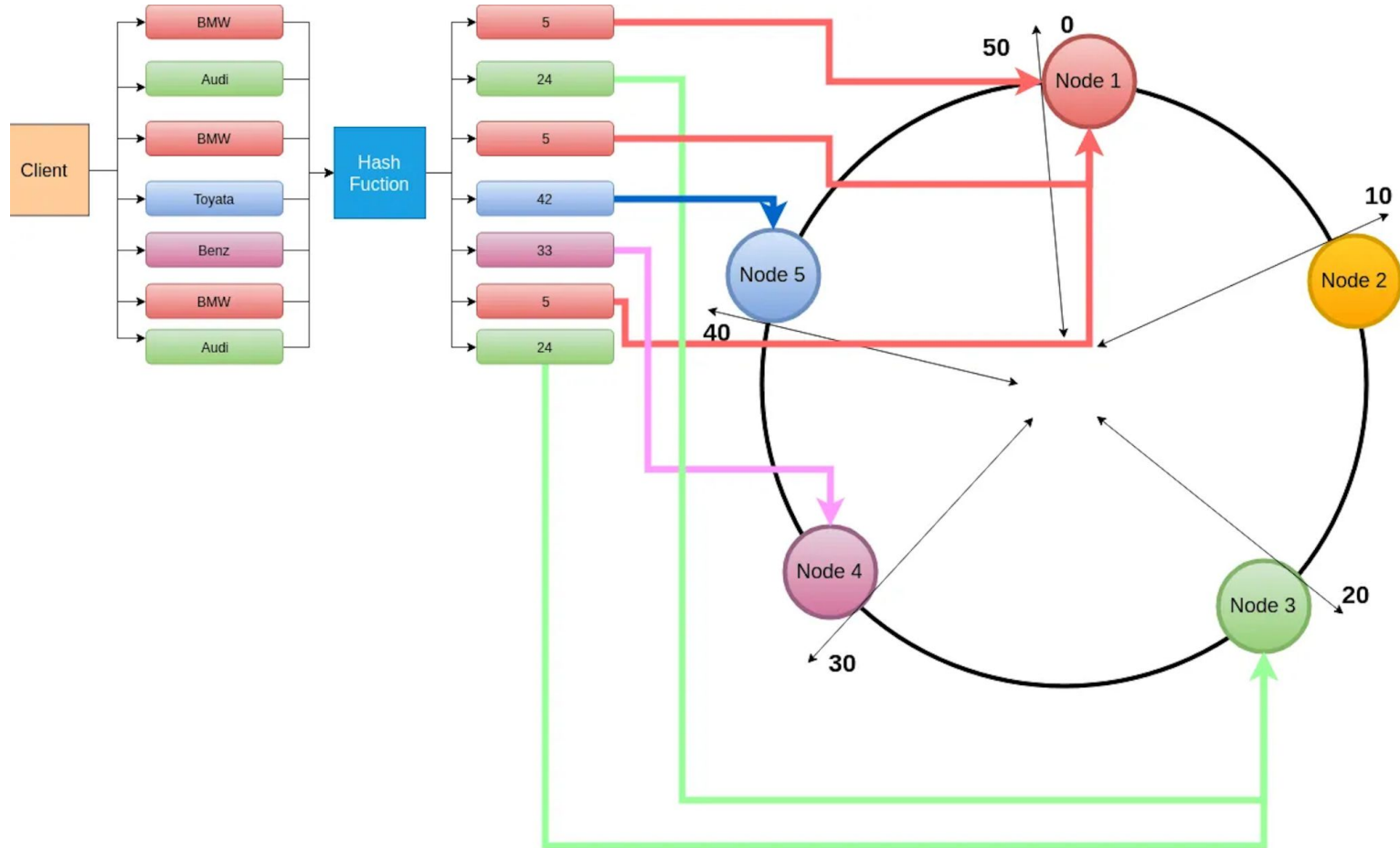
Audi -> Hash Function -> 31

Benz -> Hash Function -> 25

Data Partitioning & Token



Data Partitioning & Token



VNodes in Cassandra

In a traditional configuration, a single node is responsible for a range of tokens, splitting the token space between nodes. For example, with a three-node cluster, the first node could be responsible for token values 1 to 100, the second for 101 to 200, and the third for 201 to 300. This is called "vnode=1" configuration, which means each node has only one token range.

However, this can lead to problems when adding or removing nodes, because moving a large range of tokens can be a heavy operation. It also doesn't provide very good data distribution if some ranges of primary keys are more heavily used than others.

That's where Virtual Nodes, or "vnodes", come in. Instead of each node having a single token range, with vnodes each node can have multiple token ranges. For example, the first node might be responsible for token values 1 to 50 and 151 to 200, the second for 51 to 100 and 201 to 250, and the third for 101 to 150 and 251 to 300.

This gives a few advantages:

- **Better distribution of data and load:** Because each node handles multiple ranges, the data and load can be spread more evenly.
- **Easier to add and remove nodes:** When a new node is added, it takes over some of the token ranges from existing nodes, and it's easier to move several small ranges than one large range.

Now let's consider an example of how vnodes help in distributing data. Assume we have 6 vnodes and 3 physical nodes in our cluster. Each physical node will hold 2 vnodes.

Let's say that the entire token range of our system is from 0 to 300 (this is a simplification, as in reality the token range is based on the hash function used, which typically produces a much larger range, but the concept is the same). We can divide this entire range over our 6 vnodes:

- vnode 1: 0 to 50
- vnode 2: 51 to 100
- vnode 3: 101 to 150
- vnode 4: 151 to 200
- vnode 5: 201 to 250
- vnode 6: 251 to 300

Next, we distribute these vnodes among the physical nodes:

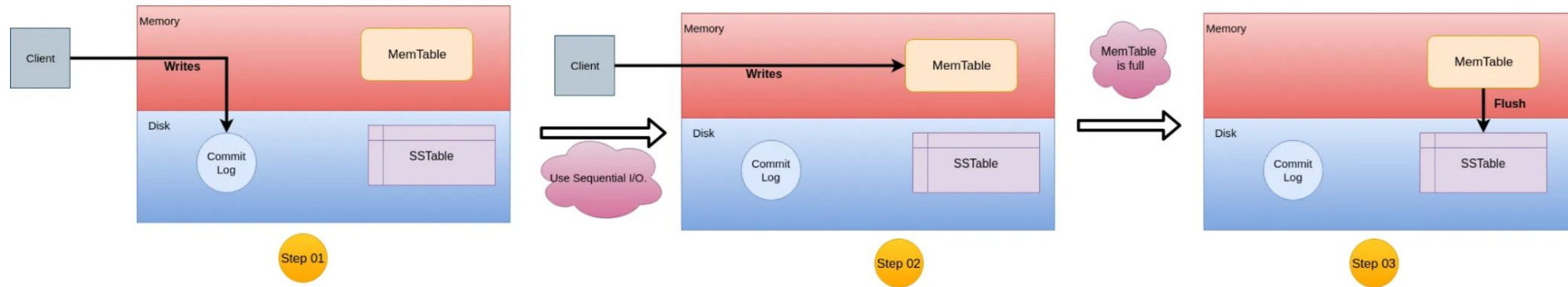
- Physical Node 1: vnode 1 (0-50) and vnode 4 (151-200)
- Physical Node 2: vnode 2 (51-100) and vnode 5 (201-250)
- Physical Node 3: vnode 3 (101-150) and vnode 6 (251-300)

So each physical node is responsible for multiple ranges in the overall token range, making data distribution more balanced and allowing the system to handle adding and removing nodes more easily.

To calculate the token range of each vnode, you simply divide the total token range by the number of vnodes. Note that, as mentioned above, in a real system the total token range would be determined by the hash function used (in Cassandra, it's Murmur3, which gives a range from -2^{63} to $+2^{63}-1$), but the principle is the same.

- **Client Request:** The client sends a write request to the coordinator node in the Cassandra cluster.
- **Commit Log Write:** The first thing the coordinator node does is write the operation to the commit log on disk. The commit log is a crash-recovery mechanism in Cassandra. It ensures that all writes are durable and can be recovered in case a node crashes or experiences a power loss.
- **Memtable Write:** After the write has been recorded in the commit log, the data is then written to an in-memory data structure called the memtable. The memtable stores data in sorted order until its size limit is reached.
- **SSTable Flush:** When the memtable is full, it is flushed to disk as an SSTable (Sorted String Table). SSTables are immutable data files which store rows of data sorted by the key.
- **Replica Write:** Depending on the configured replication factor and strategy, the write is also sent to replica nodes where steps 2, 3, and 4 are repeated. This process ensures the redundancy and high availability of data in the cluster.
- **Acknowledgement:** Once the data is written into the memtable and commit log, the node sends an acknowledgement to the coordinator node.
- **Response to Client:** After the coordinator has received acknowledgements from a sufficient number of replicas (according to the consistency level specified in the write request), it sends a success response to the client.

Write operation in Cassandra



Why Cassandra write operation is fast?

SSTables in Cassandra are indeed immutable. When there's a need to update a column, Cassandra can't modify the SSTables directly. Instead, it adds a new record. However, this process could be slow if it were to write directly to the SStable.

For better performance, Cassandra does not write updates directly to the SStable. It first writes to the commit log and the memtable, and then sends an acknowledgement to the client. The write operation is held in memory, and once it reaches a configurable limit, Cassandra flushes those changes to the SStable using sequential I/O. This approach minimizes disk I/O operations at the time of the write, contributing to Cassandra's high-speed write operations.

What happen if Cassandra machine crashes before flushing data into the SStable?

All data in Cassandra is initially written to the memtable. Suppose there's a machine crash. When the machine is rebooted and operational again, all the data previously stored in the memtable is lost.

To prevent such data loss, Cassandra uses commit logs. Every time there's a change in the memtable, Cassandra records these changes in the commit log. This allows it to keep track of memtable changes and restore any lost data when necessary.

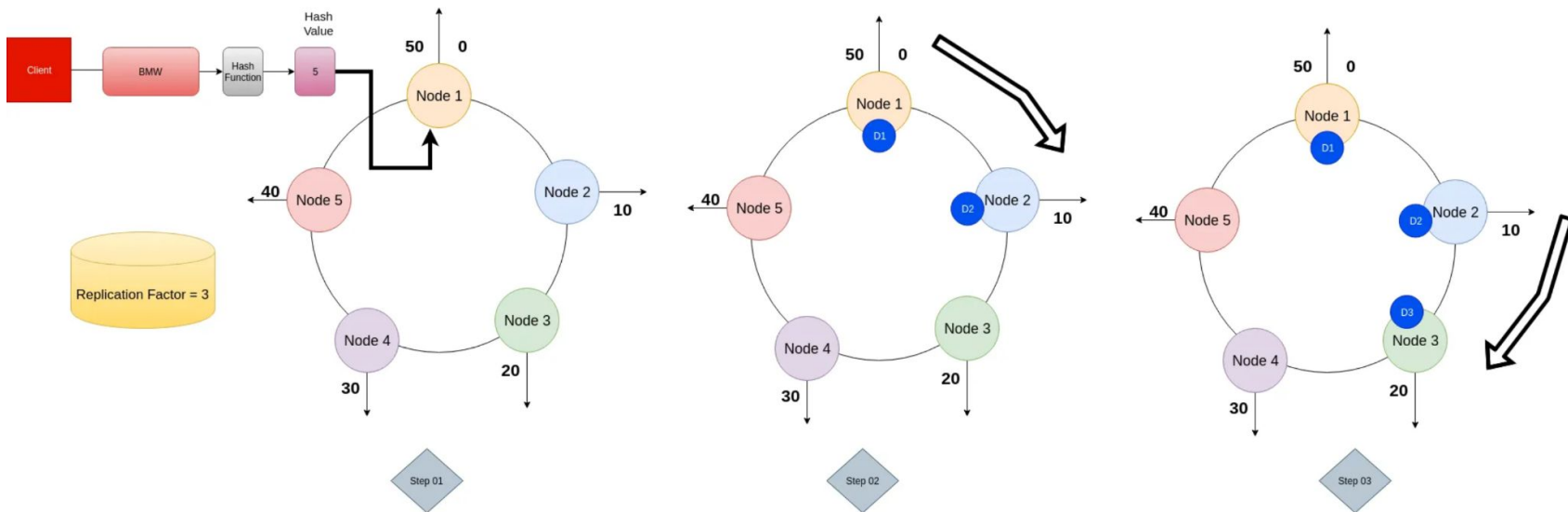
Why Cassandra writes data to commit log instead of SSTable?

The commit log in Cassandra stores changes in a single file, which greatly reduces disk I/O operations. Furthermore, even when receiving updates for multiple column families simultaneously, the disk doesn't need to perform numerous seek operations, optimizing efficiency.

Replication in Cassandra

In Apache Cassandra, the replication strategy determines the nodes where replicas are placed. The replication strategy you choose affects the distribution of data and the level of data redundancy. There are two types of replication strategies in Cassandra:

- **SimpleStrategy:** This strategy is used when you have just one data center. It places the first replica on the node selected by the partitioner. The remaining replicas are placed on the next nodes in the ring clockwise, without considering the topology (rack or data center location).



Replication in Cassandra

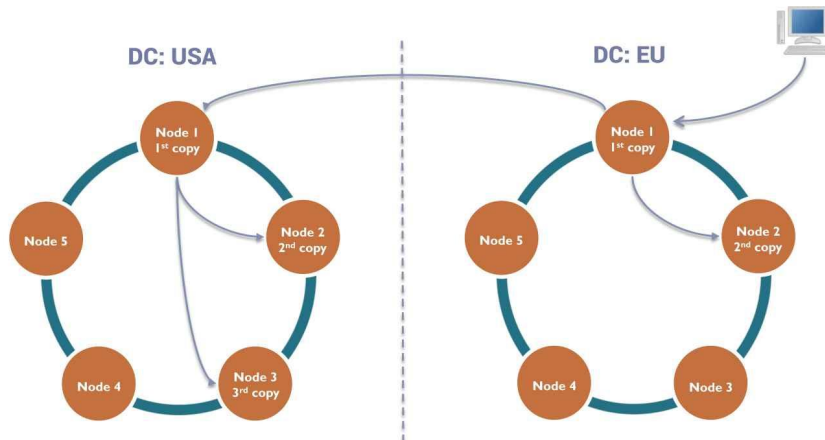
- **NetworkTopologyStrategy:** This strategy is used when you have more than one data center. It places replicas in the same data center by walking the ring clockwise until it has placed the requested number of replicas. The strategy decides how many replicas to place in each data center, and on which racks. This strategy is rack-aware and attempts to place replicas on different racks for fault tolerance.
 - **Data Center Selection:** For each write operation, Cassandra chooses a coordinator node (based on request routing or client configuration). The coordinator is responsible for handling the client's read/write requests.
 - **Replica Placement in First Data Center:** The coordinator first identifies which node in its data center owns the token range for the data to be written. This node is the primary replica. Additional replicas are placed on nodes in different racks (if available) within the same data center, up to the replication factor specified for that data center.
 - **Replica Placement in Additional Data Centers:** For the next data center, the process repeats independently. The node owning the token range for the data in the second data center becomes the primary replica for that data center. Subsequent replicas are placed on different racks within that data center, respecting the replication factor defined for that specific data center.

Replication in Cassandra

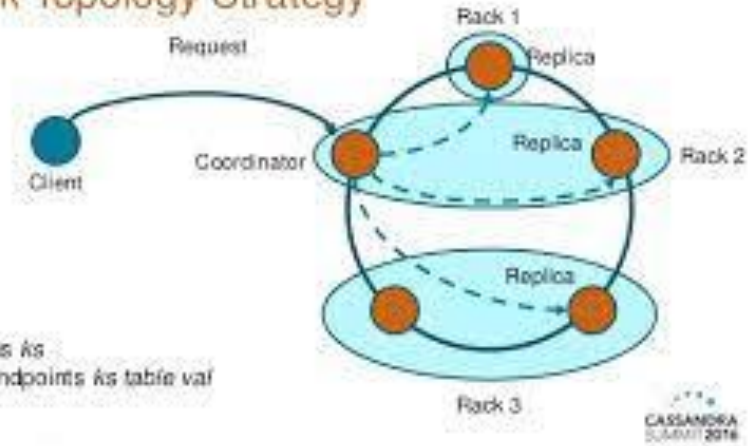
Creating A Keyspace



```
CREATE KEYSPACE johnny WITH REPLICATION =  
{'class':'NetworkTopologyStrategy', 'USA':3, 'EU': 2};
```



Network Topology Strategy

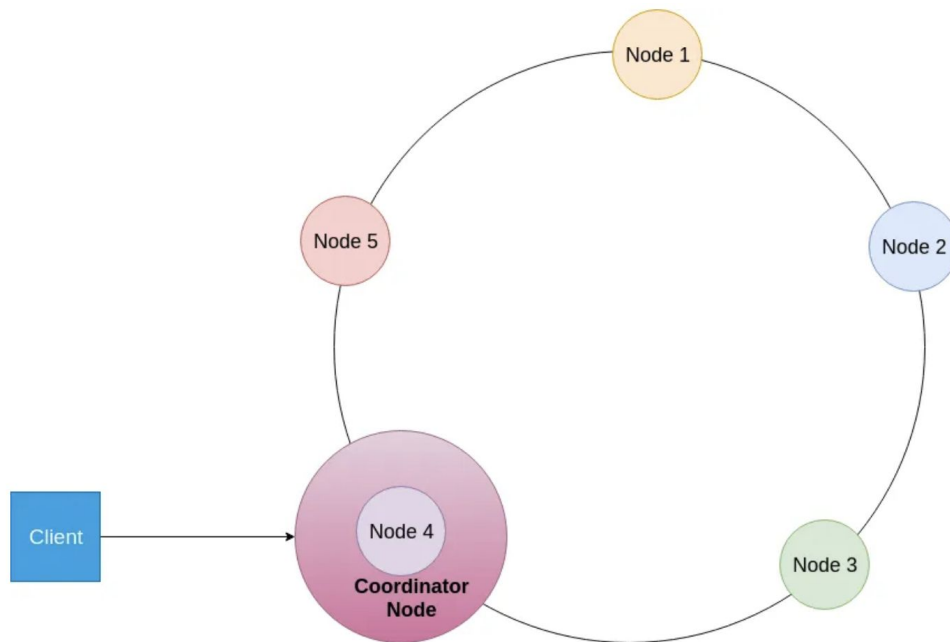


Tools:
nodetool status →
nodetool getendpoints → table val

What is Coordinator in Cassandra ?

When Cassandra client hits a **write** or **read** request on the node in the cluster. That node is called as **coordinator**. Coordinator node may be changed every time.

Coordinator is selected by the cassandra driver based on the policy, you have set. Most common policies are **DCAwareRoundRobinPolicy** and **TokenAwarePolicy**.



What is Coordinator in Cassandra ?

- **DCAwareRoundRobinPolicy:** This policy aims to prioritize nodes in the same datacenter to reduce cross-datacenter latency. It first routes requests to the local datacenter nodes and only if these nodes are unavailable or overloaded, it directs requests to nodes in other datacenters.
- **TokenAwarePolicy:** This policy directs read requests to the node that holds the replica of the data being requested, reducing the need for internode communication. It's aware of the partition key used in the query and can thus route the request directly to the nodes that own the data, resulting in increased efficiency.

Read operation in Cassandra

When read operation comes to Cassandra, that operation hit on one node. that node is coordinator node.

The row key must be supplied for every read operation. The row key is another name for the PRIMARY KEY.(partition key + clustering keys)

These are the steps when reading data from Cassandra,

1. Check the memtable
2. Check row cache, if enabled

Row cache pulls entire partitions into memory. if any part of partition has been modified , the entire cache for that row is invalidated. So invalidating big pieces of memory.

3. Checks Bloom filter
A Bloom filter in Apache Cassandra is a space-efficient, probabilistic data structure used to quickly check if an element (specifically, a row key) is likely present in an SSTable. It doesn't hold actual data but instead uses hash functions to map data to a bit array
4. Checks partition key cache, if enabled
The key cache holds the location of keys in memory on a per-column family basis. Key cache helps where a particular partition begins in the SSTable.

Read operation in Cassandra

5. Goes directly to the compression offset map if a partition key is found in the partition key cache, or checks the partition summary if not. If the partition summary is checked, then the partition index is accessed.

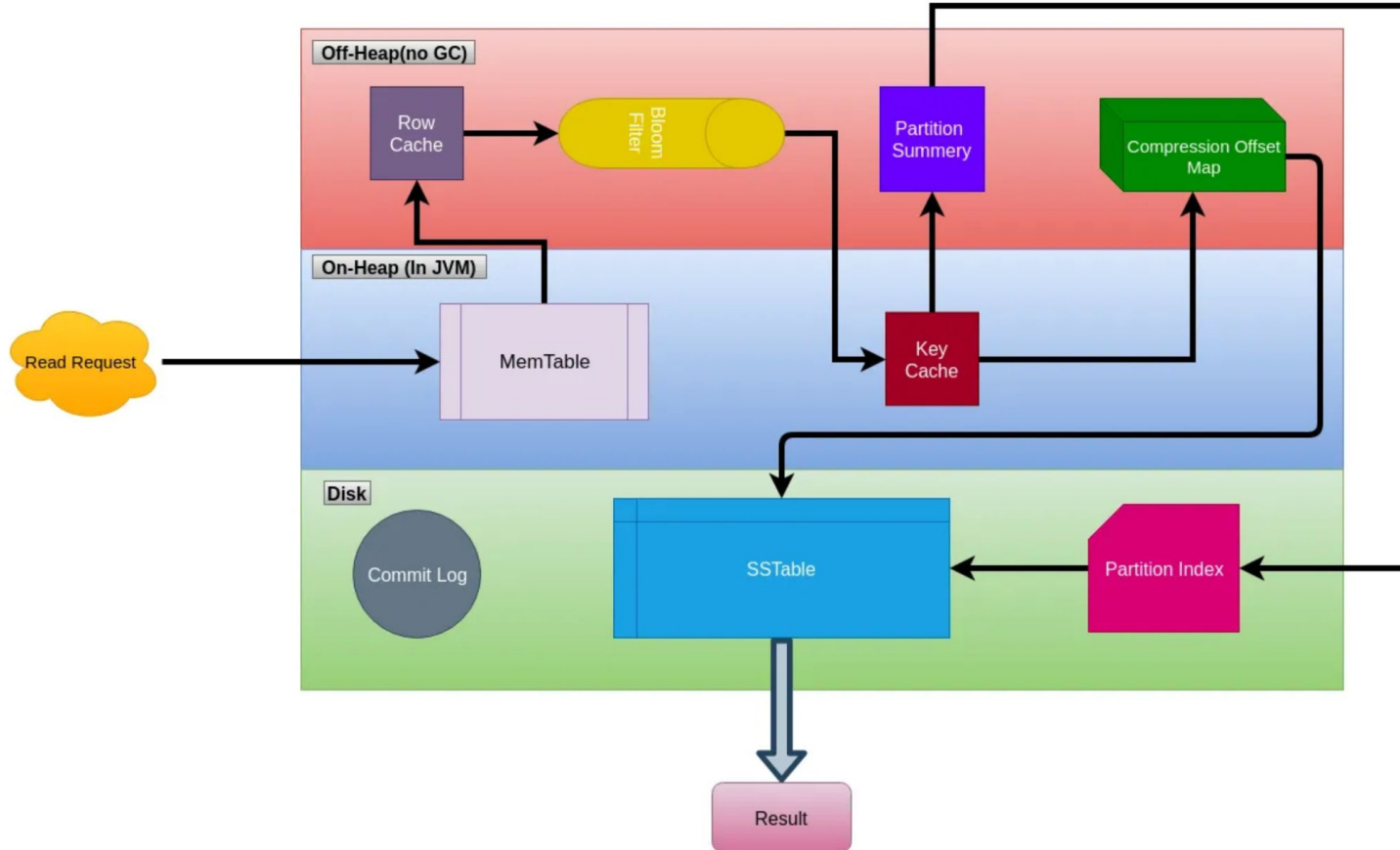
Partition summary is sampling of partition index to speedup the access to index on disk. Default sampling ratio is 128, meaning that for every 128 records for an index in index file, we have 1 record in partition summary. Each of these records of partition summary will hold key value and offset position in index.

6. Locates the data on disk using the compression offset map

Compression offset maps hold the offset information for compressed blocks.

7. Fetches the data from the SSTable on disk

Read operation in Cassandra



Compaction in Cassandra

Compaction in Apache Cassandra is the process of merging multiple SSTables (sorted string tables) into one for the purpose of improving performance and reclaiming storage space.

1. **Compaction Strategies:** Cassandra offers several compaction strategies that control how and when compaction occurs, each suited to different use cases. These include `SizeTieredCompactionStrategy`, `LeveledCompactionStrategy`, and `TimeWindowCompactionStrategy`.
 - a. **SizeTieredCompactionStrategy (STCS):** This strategy triggers compaction when a certain number of similarly-sized SSTables exist. It's most suitable for write-intensive workloads.
 - b. **LeveledCompactionStrategy (LCS):** LCS compacts SSTables into different "levels" based on their size and age, reducing read amplification at the cost of write amplification. It's most suited for read-intensive workloads.
 - c. **TimeWindowCompactionStrategy (TWCS):** TWCS groups SSTables into time windows and compacts them separately. It's designed for time-series data, where older data is rarely updated.

Gossip Protocol in Cassandra

Gossip is a peer-to-peer communication protocol used by Apache Cassandra for inter-node communication. It plays a crucial role in maintaining the health status of all nodes in a cluster. Here's a quick breakdown:

1. **Information Sharing:** Every node in a Cassandra cluster communicates with each other using Gossip to share information about themselves and about other nodes that they have gossiped about.
2. **Heartbeat:** Each node in a Cassandra cluster sends a Gossip message, or "heartbeat," to a random node every second. The heartbeat contains information about the state of the sender as well as other nodes in the cluster.
3. **Scalable and Reliable:** As a peer-to-peer protocol, Gossip is highly scalable and can function even if some nodes in the cluster are unreachable.
4. **Node State:** Gossip messages include information about a node's state (UP, DOWN), schema version, boot time, and other operational details.
5. **Failure Detection:** Gossip also helps in failure detection. If a node fails to receive a gossip message from another node within a certain period, it will consider that node as unresponsive and mark it down.
6. **Data Distribution:** Gossip is integral to data distribution in Cassandra as it helps in deciding where to read/write data, providing details about the location of replicas.

Write Consistency in Cassandra

Consistency in Apache Cassandra refers to the guarantee of how up-to-date and synchronized a row of Cassandra data is on all of its replicas.

1. Any - a write must succeed on any available node.(Highest availability)
2. One - a write must succeed on any node responsible for that row.(either primary or replica)
3. Two - a write must succeed on two nodes.
4. Quorum - a write must succeed on a quorum of replica nodes.
 - a. **quorum nodes = (replication factor/2) + 1**
5. Local Quorum - a write must succeed on a quorum of replica nodes in the same data center as the coordinator nodes.
6. All - a write must succeed on all replica nodes.(Lowest availability)

```
INSERT INTO employees (id, first_name, last_name, email)
VALUES (1, 'John', 'Doe', 'john.doe@example.com')
USING CONSISTENCY QUORUM;
```

```
UPDATE employees
USING CONSISTENCY QUORUM
SET email = 'johndoe@example.com'
WHERE id = 1;
```

Read Consistency in Cassandra

- ANY: This level is not applicable for read operations. Used only for writes.
- ONE: Only one replica needs to respond. Fastest read but might return stale data.
- TWO: Two replicas must respond.
- QUORUM: A majority of replicas (in the entire cluster) must respond. Helps balance between performance and data accuracy.
- LOCAL_QUORUM: A majority of replicas (in the local datacenter) must respond. Provides lower latency than QUORUM in multi-datacenter deployments.
- ALL: All replicas must respond. Provides the highest level of consistency but at the cost of availability if any replica is down.

```
SELECT * FROM employees WHERE id = 1 USING CONSISTENCY QUORUM;
```

Assignment Overview

The aim of this assignment is to create a **real-time** data pipeline for processing e-commerce data using Apache Kafka and Apache Cassandra. You will ingest data from a CSV file using a Kafka producer, transform the data using a Kafka consumer, and finally store the processed data in a Cassandra table.

Problem Statement

Imagine you're part of the data engineering team at a large e-commerce company. Your company receives real-time data related to customer orders. This data is crucial for various business operations, including inventory management, customer service, and business analytics.

The data is received in CSV format and needs to be processed in real-time to extract valuable insights.

Assignment Steps

1. Dataset

Load the '**olist_orders_dataset.csv**' into a pandas dataframe and examine its structure and contents.

2. Apache Kafka Setup

Install and configure Apache Kafka on your system. Create a Kafka topic, named '**ecommerce-orders**', to hold the e-commerce data.

3. Kafka Producer

Develop a Kafka producer in Python that reads the data from the pandas dataframe and publishes it to the '**ecommerce-orders**' Kafka topic. The key for each message should be a combination of the '**customer_id**' and '**order_id**' fields from the dataset.

4. Apache Cassandra Setup

Install and set up Apache Cassandra. Create a keyspace, named '**ecommerce**', for storing the e-commerce data.

5. Cassandra Data Model

Design a table, named **'orders'**, within the **'ecommerce'** keyspace. This table should reflect the schema of the incoming data and include additional columns for the derived features: **'OrderHour'** and **'OrderDayOfWeek'**. The data model should have **'customer_id'** as the partition key and **'order_id'** and **'order_purchase_timestamp'** as clustering keys.

```
CREATE TABLE ecommerce.orders (  
    order_id uuid,  
    customer_id uuid,  
    order_status text,  
    order_purchase_timestamp timestamp,  
    order_approved_at timestamp,  
    order_delivered_carrier_date timestamp,  
    order_delivered_customer_date timestamp,  
    order_estimated_delivery_date timestamp,  
    OrderHour int,  
    OrderDayOfWeek text,  
    PRIMARY KEY ((customer_id), order_id,  
    order_purchase_timestamp));
```

6. Kafka Consumer and Data Transformation

Develop a Kafka consumer (make sure to have a consumer group) in Python that subscribes to the **'ecommerce-orders'** topic. The consumer should derive two new columns **'PurchaseHour'** and **'PurchaseDayOfWeek'**, then ingest transformed data into the **'orders'** table in Cassandra.

7. Quorum Consistency

While inserting data into the Cassandra **'orders'** table, ensure that the write operations maintain quorum consistency.

8. Testing

Test your data pipeline end-to-end. Run your Kafka producer to ingest the data, then execute the Kafka consumer to process the

data and insert it into the Cassandra table. Verify the data in the Cassandra table matches the processed data and that all transformations have been executed correctly.

9. **Assignment Submission**

Submit your Python scripts for the Kafka producer and consumer, the CQL commands used to create the keyspace and table in Cassandra, and a detailed report explaining the pipeline, any challenges faced, and how they were addressed.

Grow Data Skills

Partition Key, Cluster Key and Row Key Declaration

```
CREATE TABLE employees (  
    department_id int,  
    office_id int,  
    employee_id int,  
    first_name text,  
    last_name text,  
    email text,  
    PRIMARY KEY ((department_id, office_id), employee_id, last_name)  
);
```

Cassandra Interview Questions

1. What is Apache Cassandra?

Apache Cassandra is a highly scalable, high-performance, distributed database designed to handle large amounts of data across many commodity servers, providing high availability with no single point of failure.

2. What is a token in Cassandra?

A token in Cassandra is a value that the partitioner uses to distribute data across the nodes. Each row of data is distributed across the system using the partition key and the token value.

3. What are the differences between RDBMS and Cassandra?

Cassandra is a NoSQL database while RDBMS is a SQL-based database system. RDBMS provides ACID properties, whereas Cassandra provides 'eventual consistency'. Cassandra is designed for handling big data workloads across multiple nodes without any single point of failure.

4. Can you explain what SSTable is in Cassandra?

SSTable stands for Sorted Strings Table. It's a data file to which Cassandra writes memtable data after flushing.

5. What is a Column Family in Cassandra?

Column Family in Cassandra is a structure that contains a collection of rows. Each row contains ordered columns. A column family is the equivalent of RDBMS's table.

6. Explain CAP theorem. How is it related to Cassandra?

CAP theorem stands for Consistency, Availability, and Partition Tolerance. Cassandra offers tunable consistency, which means you can balance between consistency and availability based on your requirements. By default, it provides 'eventual consistency'.

7. What is the purpose of using Cassandra Query Language (CQL)?

CQL is a language that Cassandra uses to interact with the database. It is similar to SQL, making it easier for developers with SQL background to adopt Cassandra.

8. What is tunable consistency in Cassandra?

Tunable consistency is a fantastic feature of Cassandra. It allows users to select the consistency level best suited for their use case. Consistency can be chosen on a per-operation basis.

9. What are some of the key features of Cassandra?

Some key features of Cassandra include fault tolerance, decentralized, elastic scalability, tunable consistency, and schema-free design.

10. What is a keyspace in Cassandra?

A keyspace in Cassandra is a namespace that determines how data is replicated on nodes. It's the container for your application data, similar to a schema in a relational database.

11. What are the different types of Replication Strategies in Cassandra?

There are two types of replication strategies in Cassandra: SimpleStrategy (used for a single data center setup) and NetworkTopologyStrategy (used for multiple data centers).

12. Explain Compaction in Cassandra.

Compaction is a process in Cassandra that reclaims the unused disk space, deletes the obsolete data, and merges the different copies of the same row by taking the most recent update made to a column.

13. What is the Bloom filter in Cassandra?

A Bloom filter is a space-efficient probabilistic data structure that is used to test whether an element is a member of a set. Cassandra uses Bloom filters to save IO when performing a key lookup: each SSTable has a Bloom filter associated with it that checks if any data for the requested row exists in the SSTable before doing disk I/O.

14. What is Snitch in Cassandra?

A snitch in Cassandra determines which data centres and racks nodes belong to. They inform Cassandra about the network topology so that requests are routed efficiently and allows Cassandra to distribute replicas by grouping machines into data centres and racks.

15. What are Cassandra-Stress tools and its usage?

Cassandra-Stress is a tool that allows you to stress test your Cassandra cluster. It provides a variety of options to simulate different workloads and read and write rates, thus enabling you to understand how your cluster would behave under different scenarios.

16. What is Gossip protocol in Cassandra?

Gossip is a peer-to-peer communication protocol in which nodes periodically exchange state information about themselves and about other nodes they know about. This ensures that every node in the cluster knows about every other node.

17. What is a Commit Log in Cassandra?

The commit log is a crash-recovery mechanism in Cassandra. Every write operation is written to the commit log before it is processed.

18. What is Read Repair in Cassandra?

Read Repair is a process where Cassandra performs consistency checks on a read to make sure the returned data is consistent across replicas.

19. How is data stored in Cassandra?

Data in Cassandra is stored as bytes on disk. Each row of data is stored separately and can have different numbers of columns. Columns are sorted by name.

20. Explain the role of the Coordinator node in Cassandra.

The Coordinator is the node that a client connects to and communicates with. The Coordinator node is responsible for routing the client's request to the appropriate replicas.

21. What is the Hinted Handoff in Cassandra?

Hinted Handoff is a process of handling write requests when a replica node is down or not responding. The coordinator node writes a hint indicating that a write needs to be replayed to the unavailable node.

22. What is Apache Cassandra's write pattern?

Cassandra's write pattern is referred to as "write anywhere" pattern as it allows writes to happen on any node in a cluster.

23. What is Lightweight Transaction in Cassandra?

Lightweight transactions in Cassandra use the Paxos consensus protocol to ensure linearizability i.e., a guarantee of consistency in a distributed system.

24. What is a Composite Key in Cassandra?

A composite key in Cassandra is a key that consists of multiple data elements (or columns), grouped together to create a single key.

25. Can you explain how Cassandra handles data modifications?

Cassandra writes data modifications to a commit log on disk first. Then, the changes are logged in the memtable. When the memtable is full, it is flushed to disk in an SSTable data file.

26. What are the different types of tombstones in Cassandra?

There are three types of tombstones in Cassandra: row tombstones, range tombstones, and cell tombstones. They're used to mark deleted data.

27. What is a Super Column in Cassandra?

A super column in Cassandra is a special column, therefore, it is a key-value pair where the key is a string, and the value is a map of other key-value pairs.

28. Explain the role of Memtable in Cassandra.

A Memtable is an in-memory write-back cache of data rows that Cassandra looks up during reads and updates with write operations. When a Memtable gets full, it gets flushed to disk as an SSTable.

29. What is meant by 'consistent hashing' in Cassandra?

Consistent hashing is a special kind of hashing technique that changes a minimal amount when the number of servers change. In Cassandra, consistent hashing is used to distribute data across different nodes.

30. What is Time to Live (TTL) in Cassandra?

TTL is a feature that allows data to expire after a specified period of time. It is set on a per column basis and its value is the number of seconds since the column was created.

31. What are seeds in Cassandra?

Seeds are special nodes used when a node joins a cluster. They are contacted for information about the cluster, and they serve as a communication point during startup.

32. What is the role of the Partitioner in Cassandra?

The Partitioner is responsible for distributing the data across the nodes in the cluster. It does this by determining which node a particular row of data goes to based on the partition key.

33. What is the difference between deleting and expiring in Cassandra?

Deleting in Cassandra means marking a cell as deleted, which Cassandra achieves by using tombstones. Expiring, on the other hand, is related to the TTL (Time To Live) property of a cell. When a cell is created with a TTL value, the cell will automatically be marked as deleted after the TTL has elapsed.

34. What are Repair and Anti-Entropy in Cassandra?

Repair in Cassandra refers to the process of synchronizing data across replicas. Anti-entropy is the process of ensuring that the data in a distributed system, like Cassandra, remains consistent across all nodes.

35. What is a Materialized View in Cassandra?

Materialized views are used to handle the denormalization of data in Cassandra. It's essentially a view of a base table's data that has a different primary key, and potentially, different columns as well.

36. What is the role of CQLSH in Cassandra?

CQLSH is a Python-based command-line tool, and it provides a prompt where you can interact with Cassandra using CQL (Cassandra Query Language).

37. Explain the difference between a node, a cluster, and a data center in Cassandra.

A node is a single machine running Cassandra. A cluster is a collection of nodes, and it can span across multiple data centers. A data center is a collection of related nodes and is usually tied to a physical location.

38. What is a counter column in Cassandra?

A counter column is used to store a distributed counter value, i.e., a long integer value that can be increased (but not decreased) arbitrarily.

39. Explain what "write heavy" and "read heavy" means in the context of Cassandra.

"Write heavy" and "read heavy" refer to the types of operations that are more prevalent. If you are mostly storing data, then your usage is write-heavy. If you're mostly fetching data, then your usage is read-heavy. Cassandra is often a good choice for write-heavy systems due to its log-structured storage engine.

40. What are dynamic columns in Cassandra?

Dynamic columns in Cassandra refer to the ability to have a variable number of columns in a row, unlike in traditional relational databases where the number of columns is fixed at table creation time.

41. What is a Batch in Cassandra?

A Batch in Cassandra allows you to execute multiple DML statements (like Insert, Update, Delete) as a single atomic operation.

42. What is Rack Awareness in Cassandra?

Rack Awareness is about understanding the physical layout of the nodes in the cluster in terms of racks and data centres. It allows Cassandra to intelligently place replicas to ensure data safety.

43. What is a Secondary Index in Cassandra?

Secondary Indexes in Cassandra are used to support queries on non-primary key columns. They are local to each node, and can be created on any non-primary key column.

44. What is the role of the Cassandra.yaml file?

The `cassandra.yaml` is the main configuration file of Cassandra which holds settings like cluster name, seed nodes, data file directories, etc.

45. What are Collection data types in Cassandra?

Collection data types in Cassandra allow you to store multiple values in a single column. Cassandra supports three types of collections: Lists, Sets, and Maps.

46. What is Paxos in Cassandra?

Paxos is a consensus protocol used in Cassandra for lightweight transactions. It's used to ensure that concurrent transactions don't conflict with each other.

47. What is a wide row in Cassandra?

A wide row in Cassandra is a row that has many columns. Cassandra is particularly well-suited for wide rows due to its log-structured storage engine.

48. Explain the difference between 'QUORUM' and 'LOCAL_QUORUM' consistency levels.

`QUORUM` requires a quorum of replicas in all datacenters, while `LOCAL_QUORUM` requires a quorum only in the local datacenter.

49. What are the limitations of using Secondary Indexes in Cassandra?

Secondary indexes in Cassandra can lead to increased storage costs and slower write speeds. They are also not well-suited for high cardinality columns, and performing range queries on a secondary index can be inefficient.

50. What is a Thrift in Cassandra?

Thrift is the name of the RPC (remote procedure call) framework that was used by Cassandra for client communications in earlier versions. It has been largely replaced by the native protocol in more recent versions of Cassandra.

51. Explain the differences between 'ONE', 'TWO', 'THREE', and 'ALL' consistency levels in Cassandra.

These consistency levels refer to the number of replicas that must respond before a read or a write operation is considered successful. ONE requires a response from one replica, TWO requires two replicas, and so on. ALL requires all replicas to respond.

52. What are prepared statements in Cassandra and why would you use them?

Prepared statements are pre-compiled CQL queries that you can reuse with different values for faster executions and to prevent SQL injection attacks.

53. What is 'tombstone garbage collection grace seconds'?

This is a setting in the `cassandra.yaml` file that specifies how long to wait before garbage collecting tombstones (deletion markers). Its default value is 10 days.

54. What is a Super Column Family in Cassandra?

A Super Column Family in Cassandra is a special column family that contains super columns. Each super column contains one or more columns. Super column families are deprecated in recent versions of Cassandra.

55. How does Cassandra handle concurrent writes?

Cassandra uses a timestamp value attached to each write operation to reconcile concurrent writes. The writer with the latest timestamp wins.

56. What is the purpose of the System Keyspace in Cassandra?

The System Keyspace in Cassandra is used to store internal data such as schema metadata and bootstrap tokens.

57. What are some use cases where you would not want to use Cassandra?

Some cases where Cassandra might not be the best choice include: when your data set is small and fits comfortably on a single server, when you need ACID transactions with rollback capabilities, or when you need to perform complex queries or joins.

58. What is a Column Family Store in Cassandra?

A Column Family Store in Cassandra is a type of data structure where data is stored in column families, similar to tables in a relational database. A column family store is designed to handle very large amounts of data spread out across many commodity servers.

59. How can you secure your Cassandra deployment?

You can secure your Cassandra deployment by using features such as internal or external authentication and role-based access control, client-to-node encryption, or node-to-node encryption.

60. What is the Hinted Handoff in Cassandra?

Hinted Handoff is a feature in Cassandra where if a write operation cannot be performed on a node because it is down, a hint is stored at a live node. Once the failed node recovers, the hint is used to replay the write operation. This ensures high write availability and eventual consistency.

61. How does Cassandra handle conflicts during replication?

Cassandra handles conflicts using a "last-write-wins" approach based on timestamps. The write with the latest timestamp will win.

62. What is the purpose of Cassandra's Read Repair mechanism?

Read Repair is a process in Cassandra that checks and resolves inconsistencies between replicas during a read request. It's part of Cassandra's anti-entropy operations.

63. How does the Gossip protocol work in Cassandra?

Gossip is a peer-to-peer communication protocol used by Cassandra nodes to exchange location and state information about themselves and about other nodes they have gossiped about.

64. What's the difference between Levelled Compaction and Size Tiered Compaction?

Levelled Compaction is good for write-intensive tasks. It reduces disk space usage by continually compacting SSTables into a series of levels. Size Tiered Compaction, on the other hand, is more suited for read-heavy tasks as it compacts SSTables of approximately the same size.

65. What is a Bloom Filter and how does it work in Cassandra?

A Bloom Filter is a data structure used to test whether an element is a member of a set. It's used in Cassandra to check if any data exists for a particular row on the SSTable, without needing to read the disk.

66. How does Cassandra ensure Durability?

Cassandra ensures Durability by immediately writing all writes to a commit log on disk before acknowledging the write to the client. The data is also eventually written to SSTables on disk.

67. What is the purpose of the Commit Log in Cassandra?

The Commit Log is a crash-recovery mechanism in Cassandra. Every write operation is written to the Commit Log before it is acknowledged.

68. How can you model time-series data in Cassandra?

Time-series data can be modelled in Cassandra using a wide-row model, with the time or event identifier as the column name, and the event details as the column value.

69. Explain Lightweight Transactions in Cassandra.

Lightweight Transactions in Cassandra use the Paxos consensus protocol to ensure linearizable consistency, i.e., to ensure that there are no conflicts in case of concurrent writes.

70. How does Cassandra handle data compression?

Cassandra can compress SSTable data files using different algorithms like Snappy or LZ4. Compression can be configured per table and it helps save space and reduce the amount of IO.

71. Why does Cassandra not support joins?

Cassandra doesn't support joins due to its distributed nature. Performing joins across large amounts of data distributed across many nodes would be inefficient and slow.

72. What is Eventual Consistency in Cassandra?

Eventual Consistency in Cassandra means that updates to a replicated piece of data will eventually reach all replicas. It's a compromise to provide high availability and partition tolerance.

73. What is the concept of 'Tunable Consistency' in Cassandra?

Tunable Consistency in Cassandra allows the user to decide how many replicas need to agree on a read or write operation before it is considered successful. This provides a balance between consistency, availability, and performance.

74. How does data distribution work in multi-datacenter deployments of Cassandra?

In multi-datacenter deployments, Cassandra uses NetworkTopologyStrategy to place replicas in the same datacenter to reduce latency, and across different racks to ensure redundancy.

75. Why is Cassandra suitable for IoT use cases?

Cassandra's distributed architecture, ability to handle large volumes of data, and support for time-series data modelling make it well-suited for IoT use cases.

76. Explain how compaction works in Cassandra.

Compaction in Cassandra is the process of merging SSTables on disk, discarding old or deleted data, to maintain efficient data storage.

77. What are the different types of keys in Cassandra and how are they used?

There are three types of keys in Cassandra: Partition Key, Composite Key, and Clustering Key. The partition key is used to distribute data across nodes. The composite key consists of the partition key and the clustering key. The clustering key determines the order of data inside a partition.

78. What are SSTables in Cassandra?

SSTables (Sorted String Tables) are immutable data files to which Cassandra writes memtables periodically or when they fill up.

79. How does Cassandra handle failures?

Cassandra uses various techniques to handle failures, including Hinted Handoff, Read Repair, and Active Anti-Entropy.

80. What is Cassandra's Snitch and what does it do?

A Snitch determines which data centers and racks nodes belong to. It informs Cassandra about the network topology so that requests can be routed efficiently.

81. What happens when you run out of disk space in Cassandra?

If a node runs out of disk space, it gets automatically marked as down and cannot receive write requests until sufficient space is made available.

82. What is Apache Cassandra's strategy for handling data evictions?

Cassandra doesn't support data eviction out of the box. If you want to expire data, you can set a TTL (Time To Live) while writing the data, after which the data is automatically deleted.

83. What happens when a Cassandra node goes down during a write operation?

If a node goes down during a write operation, the write can still succeed if the consistency level allows for it. The failed write is stored as a hint on a live node and replayed when the downed node comes back up.

84. How can you minimize read latencies in Cassandra?

Read latencies can be minimized by using appropriate data models, correct compaction strategies, tuning Bloom filters, and by using caches like the row cache and key cache.

85. What is the impact of Consistency Level on Cassandra's performance?

A higher consistency level can result in higher latencies as more replicas need to respond. However, it guarantees higher consistency.

86. What is the purpose of Apache Cassandra's Coordinator node?

The Coordinator node in Cassandra is the node that acts as a proxy between the client application and the nodes that own the data being requested.

87. How can you mitigate the impact of "wide rows" in Cassandra?

Wide rows can be mitigated by using appropriate compaction strategies and by careful query design. Fetching too many columns from a wide row in a single query can lead to large amounts of data being read from disk and increased latencies.

88. What is vnode and what is its purpose in Cassandra?

A vnode, or virtual node, is a subdivision of a node in the Cassandra ring. Instead of a node being responsible for a single range of data, it can be responsible for multiple ranges. This makes rebalancing easier and more efficient when nodes are added or removed.

89. How does Cassandra handle large blobs of data?

Large blobs of data can be stored as a single cell in Cassandra. However, it's not recommended as it can result in large amounts of memory usage and increased latencies. For large blobs, it's better to use an external blob store or to chunk the data into smaller pieces.

90. Explain how Tombstones work in Cassandra.

Tombstones are markers that are written when data is deleted. They are used to mark data as deleted on disk and to propagate the deletion

Scenario Based Cassandra Interview Questions

- 1. You are tasked with designing a music streaming service using Cassandra. How would you model the data to efficiently store user playlists and song metadata?**

We could have a table for songs with song_id as the primary key and columns for song metadata (like title, artist, duration, genre). For user playlists, we could use another table that uses a composite primary key of (user_id, song_id). The reason for this choice of primary key is to enable us to quickly add and remove songs from a user's playlist, as well as retrieve all the songs in a user's playlist.

```
CREATE TABLE songs (song_id UUID PRIMARY KEY, title text, artist text, duration int, genre text);  
CREATE TABLE playlists (user_id UUID, song_id UUID, added_at timestamp, PRIMARY KEY (user_id, song_id));
```

- 2. In a distributed blog platform, how would you design a schema to efficiently handle the retrieval of the latest blog posts from a particular user in Cassandra?**

we could use a TimeUUID type for the post_id. This allows us to sort posts by the time they were created. By making (user_id, post_id) the primary key for the posts table, we can quickly retrieve the latest posts for a particular user using a range query on post_id.

```
CREATE TABLE posts (user_id UUID, post_id timeuuid, title text, content text, PRIMARY KEY (user_id, post_id));
```

- 3. How would you design your Cassandra data model for a real-time chat application to ensure all messages are delivered to all participants, and older chats are retrieved quickly?**

consider using a table for messages with a primary key of (chat_id, timestamp). This would allow us to retrieve messages for a chat in chronological order. To ensure all messages are delivered to all participants, we could use a "message delivery" table that keeps track of which users have received each message.

```
CREATE TABLE messages (chat_id UUID, message_time timestamp, user_id UUID, message text, PRIMARY KEY (chat_id, message_time));
```

```
CREATE TABLE message_delivery (message_id UUID, user_id UUID, delivered boolean, PRIMARY KEY (message_id, user_id));
```

4. **If you're designing a ride-sharing application like Uber or Lyft, how would you leverage Cassandra for handling real-time locations of millions of riders and drivers?**

we could use a table for storing the location of users. The primary key could be (user_id, timestamp), which allows us to keep track of the latest location of all riders and drivers. If we needed to query locations based on proximity, we could use a technique called geohashing.

```
CREATE TABLE locations (user_id UUID, location_time timestamp, lat decimal, long decimal, PRIMARY KEY (user_id, location_time));
```

5. **You are designing a Cassandra database for a large e-commerce website. How would you create the data model to support operations such as listing all the products in a certain category, keeping track of users' shopping carts, and storing users' order history?**

we could use a table for products with category_id and product_id as a part of the primary key. This would allow us to quickly retrieve all products within a specific category.

```
CREATE TABLE products (category_id int, product_id UUID, name text, price decimal, description text, PRIMARY KEY (category_id, product_id));
```

For a shopping cart, we could use another table that tracks the items in each user's cart. Here, user_id would be a partition key, and product_id would be a clustering column, allowing us to quickly add, remove, and retrieve products for a specific user's cart.

```
CREATE TABLE carts (user_id UUID, product_id UUID, quantity int, PRIMARY KEY (user_id, product_id));
```

For order history, a separate table could be used, which allows us to quickly retrieve all the orders placed by a specific user.

```
CREATE TABLE orders (user_id UUID, order_id timeuuid, product_id UUID, quantity int, order_time timestamp, PRIMARY KEY (user_id, order_id));
```

6. **In a gaming application, player profiles are read frequently and updated occasionally, but each player's profile could be quite large. How would you design the data model in Cassandra to support this use case efficiently?**

consider using a user profile table with `user_id` as the primary key. The player's profile could contain data like the username, email, achievements, and other game-specific data. Given that profiles are read frequently but updated occasionally, we can also utilize caching techniques (like Cassandra's built-in row caching or using an external caching system like Redis) to reduce read latencies.

```
CREATE TABLE player_profiles (player_id UUID PRIMARY KEY, username text, email text, achievements list<text>);
```

- 7. You are working with a social networking site and you need to create a feature to retrieve the posts from friends that a user follows. How would you model your data in Cassandra to efficiently support this operation?**

We can create a "user_posts" table that uses (`user_id`, `post_id`) as the primary key. This enables us to quickly retrieve all posts made by a specific user.

```
CREATE TABLE user_posts (user_id UUID, post_id timeuuid, content text, PRIMARY KEY (user_id, post_id));
```

To fetch posts from friends, we would need to first fetch the list of friends for the user and then do individual reads for each friend. This operation can be made efficient by using an appropriate data model for storing friend lists and using concurrent reads to fetch posts.

- 8. You have a Cassandra cluster with multiple datacenters. How would you configure the replication strategy to ensure low latency reads and writes, as well as redundancy across different geographic regions?**

we would use the `NetworkTopologyStrategy` for replication. This strategy allows us to specify the number of replicas in each datacenter. The placement of replicas is done in a way that ensures one replica is placed in each rack, which maximizes redundancy and provides protection against rack failures. This strategy is also aware of the network topology and routes requests to the closest replicas, providing low latency reads and writes.

```
CREATE KEYSPACE mykeyspace WITH REPLICATION = { 'class' : 'NetworkTopologyStrategy', 'dc1' : 3, 'dc2' : 2 };
```

- 9. If you are developing a stock market application which needs to store and retrieve real-time and historical stock prices, how would you model the data in Cassandra?**

We need to store both real-time and historical stock prices. This is a time-series data problem and can be modeled in Cassandra by using a wide-row model. We could

use a table with (stock_symbol, timestamp) as the primary key and price as a column. This enables us to quickly insert new prices and retrieve historical price data for a specific stock.

```
CREATE TABLE stock_prices (stock_symbol text, price_time timestamp, price decimal, PRIMARY KEY (stock_symbol, price_time));
```

10. Given the task of developing a Cassandra-based backend for an IoT application that constantly receives huge amounts of sensor data, how would you design the data model and which compaction strategy would you choose to handle this use case?

Considering the constant stream of sensor data, we can model the data as time-series data. Similar to the stock market application, we could use a table with (sensor_id, timestamp) as the primary key. For the IoT application, considering the constant stream of sensor data, we can model the data as time-series data. Similar to the stock market application, we could use a table with (sensor_id, timestamp) as the primary key. The choice of compaction strategy depends on the read/write pattern. If data is mostly written and rarely read, a strategy like Leveled Compaction would be ideal. For frequent reads of recent data, Date Tiered or Time Window Compaction might be more suitable.

```
CREATE TABLE sensor_data (sensor_id UUID, recorded_at timestamp, value decimal, PRIMARY KEY (sensor_id, recorded_at));
```

11. Consider a system where you need to track user events on a website and then display a timeline of these events to the user. How would you design the Cassandra tables to efficiently handle this use case?

We could create an 'events' table with a composite primary key of (user_id, timestamp), allowing us to retrieve all events for a particular user in chronological order.

```
CREATE TABLE events (user_id UUID, event_time timestamp, event_type text, event_data text, PRIMARY KEY (user_id, event_time));
```

12. Suppose you have to implement a distributed task queue with Apache Cassandra where tasks have different priorities. How would you design this?

we could use a table 'tasks' with columns for task_id, priority, status, and other task-related data. The primary key can be (status, priority, task_id), which ensures that tasks with higher priority (lower priority number) come first.

```
CREATE TABLE tasks (status text, priority int, task_id UUID, data text, PRIMARY KEY (status, priority, task_id));
```

- 13. Imagine you're developing a movie recommendation system. How would you design a Cassandra schema to efficiently store and retrieve user ratings for various movies?**

We can create a 'ratings' table where the primary key is (user_id, movie_id), and there's a column for rating. This allows quick update and retrieval of a user's rating for a movie.

```
CREATE TABLE ratings (user_id UUID, movie_id UUID, rating int, PRIMARY KEY (user_id, movie_id));
```

- 14. You are tasked with developing an online collaborative document editing platform. How would you leverage Cassandra to store document data and handle simultaneous edits from multiple users?**

we could store each document as a collection of operations or changes, rather than the document content itself. Each operation could be an insertion, deletion, or update, and should include the position in the document and the content to be inserted or updated.

```
CREATE TABLE document_changes (doc_id UUID, change_time timestamp, user_id UUID, operation text, position int, content text, PRIMARY KEY (doc_id, change_time));
```

- 15. You have a requirement to store time-series data for an application monitoring system, where you need to track system metrics every minute. How would you model your data in Cassandra to serve this requirement?**

We could create a 'metrics' table with (system_id, metric_name, timestamp) as the primary key, allowing us to retrieve all values of a specific metric for a particular system in a given time range.

```
CREATE TABLE metrics (system_id UUID, metric_name text, recorded_at timestamp, value decimal, PRIMARY KEY ((system_id, metric_name), recorded_at));
```